

Relazione Tecnica

di Progettazione e Sviluppo

Travel Organizer Offline

App mobile per l'organizzazione dei viaggi

Giugno 2026

Gruppo

Componente 1

Componente 2

Componente 3

Indice

1	Descrizione dell'app	2
1.1	Obiettivo	2
1.2	Tipologia di utenti	2
1.3	Problema che l'app intende risolvere	2
1.4	Principali scenari d'uso	2
2	Requisiti	2
2.1	Quadro generale dei requisiti	2
2.2	Modelli dei dati (DF)	2
2.3	Funzionalità implementate (IF)	3
2.4	Feature avanzate	3
2.5	Funzionalità considerate ma non implementate	5
2.6	Limitazioni note	5
3	Progettazione dell'app	5
3.1	Struttura generale	5
3.2	Schermate e navigazione	5
3.3	Organizzazione dei dati	5
3.4	Integrità dei dati	6
4	Scelte tecnologiche	6
4.1	Framework e librerie	6
4.2	Gestione della persistenza	7
4.3	Come le feature avanzate valorizzano l'architettura locale	7
5	Implementazione	7
5.1	Organizzazione del codice	7
5.2	Gestione dello stato e della navigazione	7
5.3	Parti significative	7
5.4	Dati di esempio precaricati	8
6	Casi d'uso	8
6.1	Elenco dei casi d'uso	8
6.2	Descrizione di alcuni casi d'uso	8
7	Conclusioni	9

1 Descrizione dell'app

1.1 Obiettivo

Travel Organizer Offline è un'applicazione mobile, sviluppata in *Flutter*, che aiuta l'utente a pianificare e gestire i propri viaggi: itinerari, tappe, attività, checklist e spese. Tutte le informazioni restano disponibili **anche senza connessione di rete**, perché i dati sono salvati in un database locale sul dispositivo. L'app non utilizza API remote né un backend: è progettata per funzionare al 100% offline.

1.2 Tipologia di utenti

L'applicazione si rivolge a chiunque organizzi viaggi – viaggiatori abituali, studenti, famiglie – e desideri uno strumento semplice per tenere sotto controllo programma, bagaglio e budget anche dove la rete non è disponibile (aereo, estero, zone remote).

1.3 Problema che l'app intende risolvere

Le informazioni di un viaggio sono spesso frammentate tra note, chat e siti diversi e richiedono una connessione per essere consultate. Travel Organizer Offline centralizza tutto in un'unica app consultabile offline, evitando la dipendenza dalla rete e la dispersione dei dati.

1.4 Principali scenari d'uso

- **Pianificazione:** l'utente crea un viaggio (scegliendo tag e modalità di viaggio), lo suddivide in tappe e associa a ciascuna le attività con orari e luoghi.
- **Preparazione:** genera la lista valigia organizzata in sacchetti in base ai tag del viaggio e compila le checklist dei documenti.
- **Controllo spese:** registra spese previste ed effettive e monitora il budget tramite la barra di salute e il calcolatore del ritmo di spesa.
- **In viaggio:** consulta offline la timeline della giornata e i tempi liberi; riceve un promemoria 30 minuti prima di ogni attività.
- **Ricordi:** per ogni attività scrive un pensiero nel diario e allega una foto dalla galleria del telefono.
- **Emergenza:** consulta i numeri di emergenza del paese di destinazione e i propri dati medici, sempre disponibili offline.
- **Condivisione:** esporta un viaggio in JSON e lo invia a un amico, che può reimportarlo nella propria app.

2 Requisiti

2.1 Quadro generale dei requisiti

2.2 Modelli dei dati (DF)

- **DF-1.1 Viaggio:** id, titolo, destinazione, dataInizio, dataFine, descrizione, stato, budget, partecipanti, note, *tag*, *modalità di viaggio*, dataCreazione.
- **DF-1.2 Tappa:** id, viaggioId, titolo, data, località, descrizione, ordine, note.
- **DF-1.3 Attività:** id, viaggioId, tappaId, titolo, descrizione, dataOra, luogo, categoria, costoPrevisto, stato, note, *nota diario*, *percorso foto*.
- **DF-1.4 Checklist:** id, viaggioId, tappaId, titolo, descrizione, elementi.
- **DF-1.5 Elemento checklist:** id, checklistId, titolo, completato, ordine, *sacchetto (categoria)*.

Categoria	Codice	Descrizione	Priorità
Dati	DF-1	Modellazione e persistenza (Viaggi, Tappe, Attività, Checklist, Spese, Impostazioni)	Must
Funzione	IF-1	Gestione dei Viaggi	Must
Funzione	IF-2	Gestione delle Tappe/Giornate	Must
Funzione	IF-3	Gestione delle Attività e dell'itinerario	Must
Funzione	IF-4	Gestione delle Checklist e informazioni utili	Must
Funzione	IF-5	Gestione delle Spese e del budget	Must
Funzione	IF-6	Ricerca, filtri e ricerca globale	Should
Funzione	IF-7	Riepiloghi, statistiche e calendario	Should
Funzione	IF-8	Feature avanzate	Should
Flusso	BF	Flussi inter-modulo	Must

- **DF-1.6 Spesa:** id, viaggioId, tappaId, attivitàId, titolo, importo, categoria, data, metodoPagamento, stato (prevista/effettiva), note.
- **DF-1.7 Impostazioni:** archivio chiave-valore per i dati che non appartengono a un viaggio (es. dati di emergenza ICE).

2.3 Funzionalità implementate (IF)

- **IF-1 Viaggi:** elenco, dettaglio, inserimento, modifica, eliminazione, duplicazione, assegnazione di **tag** e di una **modalità di viaggio** (auto, aereo, treno, nave, crociera, bicicletta, autobus, a piedi). Lo stato è calcolato dalle date.
- **IF-2 Tappe:** elenco, inserimento, modifica, eliminazione, riordino con drag & drop e accesso alle attività associate.
- **IF-3 Attività:** inserimento, modifica, eliminazione, associazione a viaggio/tappa, cambio di stato, categoria, data/ora, luogo, costo previsto e sezione **diario** (nota + foto).
- **IF-4 Checklist:** creazione, gestione e spunta degli elementi, barra di avanzamento, duplicazione e **lista valigia organizzata in sacchetti**.
- **IF-5 Spese:** inserimento, modifica, eliminazione, spese previste/effettive, confronto col budget, barra di salute e **calcolatore del ritmo di spesa**.
- **IF-6 Ricerca e filtri:** filtri per stato/categoria/data/importo su viaggi, tappe, attività e spese, più una **ricerca globale (Spotlight)** che cerca contemporaneamente in tutte le entità.
- **IF-7 Statistiche e calendario:** riepiloghi, grafici e una vista **calendario mensile** con marker per viaggi e attività.
- **IF-8 Feature avanzate:** descritte nella Sezione 2.4.

2.4 Feature avanzate

La traccia richiede almeno due feature avanzate; il progetto ne integra molte, tutte effettivamente funzionanti e collegate al dominio applicativo. Vengono qui descritte in dettaglio.

1. Packing list intelligente con tag e sacchetti

A ogni viaggio si possono assegnare dei tag (Mare, Montagna, Città, Estero, Business, Trekking, Inverno). Un **catalogo predefinito nel codice** (matrice tag → oggetti, completamente offline) associa a ogni tag un insieme di oggetti adatti: ad esempio *Mare* → Costume, Crema

solare; *Estero* → Passaporto, Adattatore prese. Generando la lista valigia, gli oggetti di base e i suggerimenti dei tag vengono uniti senza duplicati e **organizzati in “sacchetti”** (Abbigliamento, Documenti, Elettronica, Igiene, Medicinali, Varie). Ogni sacchetto ha una propria barra di avanzamento (es. “Elettronica: 3/5”), così la valigia non diventa una lunga lista caotica.

2. Timeline visuale con tempi liberi

L’itinerario è mostrato come linea del tempo con **blocchi colorati in base alla categoria** dell’attività (es. pasti = arancione, spostamenti = blu). Tra un’attività e la successiva l’app calcola i **periodi liberi** e li evidenzia con un badge discreto (es. “Hai 3h di tempo libero”), aiutando a capire dove c’è spazio per altre attività.

3. Gamification del budget e calcolatore di sbilanciamento

La salute del budget è rappresentata da una barra che cambia colore in modo progressivo (verde → giallo → rosso) all’aumentare della spesa. Inoltre un **calcolatore di ritmo di spesa** confronta quanto è stato speso con quanto ci si aspetterebbe in base ai giorni di viaggio trascorsi: se, ad esempio, con un budget di 700€ per 7 giorni (100€/giorno) al terzo giorno si sono spesi 500€, l’app segnala lo sforamento e **ricalcola il budget giornaliero** per i giorni rimanenti. Quando si inserisce una spesa che supera il budget residuo, compare un avviso di conferma.

4. Esporta/Importa viaggio (JSON + condivisione nativa)

Poiché non esiste un backend, la condivisione avviene in locale: un viaggio completo (con tappe, attività, checklist e spese) viene **serializzato in JSON** e condiviso tramite il menu nativo del sistema operativo (`share_plus`). Un altro utente può incollare il testo e **reimportarlo**; in fase di import gli identificatori vengono rigenerati e le relazioni ricollegate, così da non creare conflitti con i dati esistenti.

5. Calendario mensile interattivo

Una vista calendario (`table_calendar`) mostra i **marker** sui giorni che contengono eventi: partenza e ritorno dei viaggi e attività pianificate. Selezionando un giorno si vede l’elenco degli eventi e si può aprire direttamente il viaggio collegato.

6. Diario di viaggio con foto locali

Ogni attività ha una sezione “Diario” in cui l’utente scrive un pensiero e **allega una foto** scelta dalla galleria o scattata sul momento (`image_picker`). Il percorso locale dell’immagine viene salvato nel database e la miniatura compare nell’elenco delle attività della tappa. L’app diventa così non solo un organizzatore ma anche un diario personale.

7. Notifiche locali per le attività

Quando un’attività ha un orario, l’app **programma una notifica locale** (`flutter_local_notifications` con `timezone`) che avvisa l’utente **30 minuti prima**, anche ad app chiusa, sfruttando le API native del dispositivo senza alcun server. La notifica viene riprogrammata se l’attività cambia e annullata se viene eliminata.

8. Sezione “In caso di emergenza” (ICE) dinamica

Una scheda dedicata, accessibile rapidamente anche offline, raccoglie i **dati medici essenziali** dell’utente (gruppo sanguigno, allergie, patologie, contatti) salvati in locale. In base alla destinazione del viaggio in corso o del prossimo, l’app **deduce automaticamente il paese** e mostra

i relativi numeri di emergenza standard (es. 112 in Europa, 911 negli Stati Uniti, 110/119 in Giappone), presi da un **dizionario interno al codice**.

9. Ricerca globale “Spotlight”

Una barra di ricerca unica cerca in modo predittivo, mentre l’utente digita, **contemporaneamente** tra titoli dei viaggi, tappe, nomi delle attività e titoli/note delle spese, mostrando i risultati **raggruppati per categoria**.

10. Duplicazione e modalità di viaggio

È possibile duplicare viaggi, tappe e checklist, e indicare la modalità con cui si svolge il viaggio (auto, aereo, treno, nave, crociera, bicicletta, ecc.) con un selettore a pulsanti.

2.5 Funzionalità considerate ma non implementate

- Simulatore di cambio valuta con tasso manuale per viaggio.
- Salvataggio di “giornate tipo” come modelli riutilizzabili.
- Temi grafici dinamici in base alla destinazione.

2.6 Limitazioni note

- La gestione del cambio valuta non è inclusa: gli importi sono in un’unica valuta (Euro).
- L’import di un viaggio avviene incollando il testo JSON (non è prevista la lettura diretta di un file).
- Le notifiche locali usano il fuso orario italiano (*Europe/Rome*) e non sono disponibili nella versione Web.
- I dati sono locali al dispositivo: non esiste sincronizzazione su cloud.

3 Progettazione dell’app

3.1 Struttura generale

L’app adotta un’architettura a livelli che separa nettamente **interfaccia**, **stato** e **accesso ai dati**:

UI (Schermate) → Provider (Stato) → Repository → Database SQLite

Le schermate non conoscono il database, ma dialogano con i provider; i provider usano i repository, che incapsulano le query SQL. Servizi trasversali (notifiche, ricerca globale, import/export) sono isolati in classi dedicate.

3.2 Schermate e navigazione

La navigazione principale avviene tramite una *Bottom Navigation Bar* a **quattro sezioni**: **Viaggi**, **Calendario**, **Statistiche** ed **Emergenza**. Dalla lista dei viaggi si apre il dettaglio (passaggio dell’id alla schermata successiva), organizzato in sei schede: Tappe, Attività, Checklist, Spese, Timeline e Valigia. Da ciascuna scheda si raggiungono i form di inserimento/modifica. Le transizioni usano *Navigator* e *MaterialPageRoute*, con passaggio esplicito dei dati.

3.3 Organizzazione dei dati

Lo schema relazionale prevede una tabella per ciascuna entità, con vincoli di chiave esterna ed **eliminazione a cascata**: cancellando un viaggio vengono rimossi automaticamente tappe, attività, checklist (con i relativi elementi) e spese collegate.

Tabella	Relazione	Note
trips	–	Entità principale (con tag e modalità)
stages	trips (1:N)	ON DELETE CASCADE
activities	trips/stages	CASCADE su trip, SET NULL su stage; diario e foto
checklists	trips/stages	CASCADE su trip
checklist_items	checklists (1:N)	ON DELETE CASCADE; sacchetto
expenses	trips (1:N)	ON DELETE CASCADE
settings	–	Archivio chiave-valore (dati ICE)

Lo schema è versionato: le **migrazioni** (`onUpgrade`) aggiungono in modo incrementale le colonne delle nuove funzionalità (tag, modalità, sacchetti, diario) e la tabella delle impostazioni, senza perdere i dati esistenti.

3.4 Integrità dei dati

Oltre ai vincoli del database, l'app applica regole a livello applicativo:

- impedisce di salvare due viaggi con **date sovrapposte**;
- valida che la data di fine non preceda quella di inizio;
- richiede i campi obbligatori (titolo, destinazione, importo);
- dopo l'eliminazione di un viaggio, allinea le cache in memoria così le statistiche restano coerenti.

4 Scelte tecnologiche

4.1 Framework e librerie

È stato scelto **Flutter** (Material 3) per sviluppare con un unico codice sorgente un'app fluida e multiplatforma (Android, iOS, Web).

Libreria	Ruolo
provider	Gestione dello stato (ChangeNotifier).
sqlite / sqflite_common_ffi_web	Persistenza locale su SQLite (anche su Web via WebAssembly).
fl_chart	Grafici della sezione statistiche.
table_calendar	Calendario mensile con marker.
image_picker	Selezione/scatto foto per il diario.
flutter_local_notifications + timezone	Notifiche locali programmate.
share_plus	Condivisione tramite menu nativo (esporta viaggio).
intl	Formattazione di date e valuta in italiano.
uuid	Generazione di identificatori univoci.

4.2 Gestione della persistenza

Tutti i dati sono salvati in un database **SQLite locale**. Un **DatabaseHelper** (singleton) gestisce apertura, creazione dello schema, attivazione delle chiavi esterne e migrazioni di versione. I **Repository** incapsulano le query SQL per ciascuna entità.

4.3 Come le feature avanzate valorizzano l'architettura locale

L'assenza di servizi cloud ha spinto a sfruttare al massimo le risorse del dispositivo e un database relazionale ben progettato:

- le **chiavi esterne stabili** tra Viaggi → Tappe → Attività/Spese e l'eliminazione a cascata garantiscono dati sempre coerenti, e rendono possibili funzioni come l'export/import dell'intero grafo di un viaggio;
- la **reattività della UI** (grafici, barre di salute e statistiche che si aggiornano in tempo reale) è garantita da una gestione dello stato locale efficiente basata su **ChangeNotifier**;
- le funzioni "intelligenti" (packing list per tag, numeri di emergenza per paese) si basano su **dizionari interni al codice**, dimostrando come si possa ottenere un comportamento utile senza alcuna API esterna;
- l'integrazione con le **API native** del telefono (notifiche locali, fotocamera/galleria, menu di condivisione) aggiunge valore mantenendo il funzionamento completamente offline.

5 Implementazione

5.1 Organizzazione del codice

```
lib/  
  core/      Tema, colori, utility, cataloghi (packing, emergenza),  
             servizio notifiche  
  data/  
    database/ DatabaseHelper (schema, migrazioni)  
    models/   Trip, Stage, Activity, Checklist, Expense  
    repositories/ Accesso ai dati (query SQL) + impostazioni  
    seed/     Dati di esempio (viaggi precaricati)  
    trip_transfer.dart  Esporta/Importa viaggio (JSON)  
    global_search.dart  Ricerca globale (Spotlight)  
  providers/  Stato dell'app (ChangeNotifier)  
  features/   Schermate: trips, stages, activities, expenses, checklists,  
             timeline, packing, stats, calendar, emergency, search  
  shared/widgets/ Widget riutilizzabili (EmptyState, ConfirmDialog, ...)  
  main.dart   Avvio: localizzazione, notifiche, seed, app
```

5.2 Gestione dello stato e della navigazione

Lo stato è gestito con il pattern **Provider/ChangeNotifier**: ogni entità ha un proprio provider che mantiene i dati in memoria, esegue le operazioni tramite i repository e notifica la UI con `notifyListeners()`. La barra inferiore usa un **IndexedStack** per conservare lo stato delle sezioni.

5.3 Parti significative

- **Controllo di integrità sulle date** dei viaggi (niente sovrapposizioni), validato da test automatici.
- **Esporta/Importa**: serializzazione dell'intero grafo di un viaggio in JSON e re-importazione con rimappatura degli identificatori.

- **Servizio notifiche** robusto: protetto da guardie, diventa un “no-op” su Web o piattaforme non supportate senza bloccare l’app.
- **Gestione foto multiplatforma**: su mobile il percorso è un file, su Web un URL; un widget dedicato sceglie automaticamente come caricarlo.
- **Test**: il progetto include test automatici (avvio dell’app, integrità delle date, eliminazione, esporta/importa, dati di esempio).

5.4 Dati di esempio precaricati

Al primo avvio, se il database è vuoto, l’app inserisce **quattro viaggi di esempio completi** che mostrano tutte le funzionalità:

- **Tokyo, Barcellona e Roma** (conclusi), con attività completate, note di diario, checklist a sacchetti e spese effettive;
- **Parigi** (in programma, 27 ottobre – 1 novembre 2026), con un **itinerario completo** di cinque giornate: arrivo e Île de la Cité, Louvre e Tour Eiffel, Montmartre, Versailles, Champs-Élysées e Marais, con orari, luoghi, costi previsti, lista valigia e checklist documenti.

6 Casi d’uso

6.1 Elenco dei casi d’uso

ID	Caso d’uso	ID	Caso d’uso
UC-1	Inserisci viaggio (tag e modalità)	UC-7	Ricerca globale
UC-2	Pianifica attività	UC-8	Visualizza statistiche
UC-3	Genera packing list a sacchetti	UC-9	Visualizza calendario
UC-4	Registra spesa e controlla budget	UC-10	Esporta / Importa viaggio
UC-5	Scrivi il diario con foto	UC-11	Consulta sezione emergenza
UC-6	Ricevi promemoria attività	UC-12	Modifica o elimina elemento

6.2 Descrizione di alcuni casi d’uso

UC-1: Inserisci viaggio

Attore: Utente.

Postcondizioni: Un nuovo viaggio è salvato e mostrato nell’elenco.

Flusso base: l’utente compila titolo, destinazione, date, budget, partecipanti, **tag e modalità di viaggio**; il sistema valida i campi e le date e salva.

Alternativo: se le date si sovrappongono a un altro viaggio, il salvataggio è bloccato con un avviso.

UC-3: Genera packing list a sacchetti

Flusso base: l’utente apre la scheda Valigia e seleziona “Genera lista”; il sistema legge i tag del viaggio, recupera dal catalogo gli oggetti suggeriti e li aggiunge **organizzati in sacchetti**, ciascuno con la propria barra di avanzamento.

UC-4: Registra spesa e controlla il budget

Flusso base: l'utente inserisce una spesa; il sistema aggiorna i totali, la barra di salute del budget e il **calcolatore del ritmo di spesa**.

Alternativo: se la spesa effettiva supera il budget residuo, il sistema mostra un avviso e chiede conferma prima di salvare.

UC-5: Scrivi il diario con foto

Flusso base: nel form dell'attività l'utente scrive un ricordo e seleziona/scatta una foto; il sistema salva nota e percorso dell'immagine e mostra l'anteprima.

UC-6: Ricevi promemoria attività

Flusso base: salvando un'attività con orario futuro, il sistema programma una notifica locale 30 minuti prima; la notifica viene mostrata anche ad app chiusa.

Alternativo: se l'orario è già passato, non viene programmato nulla.

UC-9: Visualizza calendario

Flusso base: l'utente apre la scheda Calendario; il sistema mostra i marker sui giorni con eventi; selezionando un giorno vede gli eventi e può aprire il viaggio collegato.

UC-11: Consulta sezione emergenza

Flusso base: l'utente apre la scheda Emergenza; il sistema deduce il paese dal viaggio più rilevante e mostra i numeri di emergenza locali, oltre ai dati medici salvati. L'utente può modificare e salvare i propri dati.

7 Conclusioni

Travel Organizer Offline soddisfa tutte le funzionalità richieste dalla traccia – viaggi, tappe, attività, checklist e spese – aggiungendo ricerca, filtri, statistiche, calendario e numerose feature avanzate effettivamente integrate (packing list intelligente a sacchetti, gamification e ritmo del budget, timeline con tempi liberi, diario con foto, notifiche locali, sezione emergenza, ricerca globale, esporta/importa). L'architettura a livelli, la persistenza locale coerente e i controlli di integrità rendono l'app robusta, semplice da usare e completamente funzionante offline.